

Container

■ Container:

- I container isolano i processi usando funzionalità del kernel:
 - **Namespace** per creare mondi separati
 - **Cgroups** per controllare le risorse dei containers

■ namespace:

- Fusione del kernel per creare viste isolate delle risorse del sistema
- Ogni container vede propri/o:
 - processi, rete, filesystem, hostname, tabella di montaggio, utenti

■ contro cgroups:

- Limitare uso di risorse:
 - CPU, RAM, I/O disco, traffico rete
- Congelare processi
- monitorare consumo

Container

- Con i namespace si possono isolare:
 - solo i processi (PID namespace)
 - solo la rete (NET namespace)
 - solo il filesystem (MNT namespace)
 - solo gli hostname (UTS namespace)
 - solo la comunicazione IPC (IPC namespace)
 - solo gli utenti/UID (USER namespace)

Kernel Linux unico

- Namespace Container A → vede solo i suoi processi (PID)
- Namespace Container B → vede solo la sua rete (NET)
- Namespace Container C → vede solo il suo FS (MOUNT)

Container

- Con i namespace si possono isolare:
 - solo i processi (PID namespace)

Esempio

```
sudo unshare --pid --fork --mount-proc bash
```

--fork	→ avvia il comando dentro il nuovo namespace
--mount-proc	→ /proc viene rimontato nel nuovo namespace

Crea un processo figlio che entra nel namespace appena creato.
Questo processo vede solo i suoi processi.
Diventa PID 1, come in un container reale.

Quando il processo termina, il namespace viene distrutto.

Container

- Con i namespace si possono isolare:
 - solo i processi (PID namespace)

Esempio

```
sudo unshare --pid --fork --mount-proc bash
```

--fork	→ avvia il comando dentro il nuovo namespace
--mount-proc	→ /proc viene rimontato nel nuovo namespace

Crea un processo figlio che entra nel namespace appena creato.
Questo processo vede solo i suoi processi.
Diventa PID 1, come in un container reale.

Quando il processo termina, il namespace viene distrutto.

Container

- Con i namespace si possono isolare:
 - isolare hostname (UTS namespace)

Esempio

```
sudo unshare --uts --fork bash
```

Il namespace UTS isola: l'hostname e il domain name

Quindi un processo può credere di avere un hostname diverso dall'host senza modificarlo realmente.

```
hostname new-name  
hostname
```

In una nuova shell provare
hostname

Container

- Creare un mini-container:

Esempio

```
sudo unshare --pid --uts --ipc --net --mount --fork --mount-proc bash
```

Cosa isola:

PID	il processo diventa PID 1, come init
Hostname (UTS)	container può avere un nome diverso
IPC	code messaggi/semafori isolati
Rete	nessuna interfaccia dell'host, solo <code>lo</code>
Mount	tabella dei mount indipendente
/proc	vista dei processi isolata

Come si esce: `exit`

Container

- Namespace nel Kernel:

I namespace sono anonimi

Ogni namespace è rappresentato da una struttura C nel kernel:

- struct pid_namespace
- struct uts_namespace
- struct net
- struct user_namespace

Ogni processo (task_struct) contiene puntatori a queste strutture:

```
struct task_struct {  
    struct nsproxy *nsproxy; // punta a tutti i namespaces  
    ...  
};
```

```
pid_ns_for_children → (PID namespace B)  
uts_ns → (...)  
net_ns → (...)  
mnt_ns → (...)  
ipc_ns → (...)
```

Container

- Namespace nel Kernel:

Nel kernel ogni processo ha *uno* PID reale ("global PID")

I PID namespaces formano un albero (global → figlio → nipote)

Il processo *appartiene solo al nodo foglia*

Il processo vede solo il PID del suo namespace corrente

Il kernel può tradurre PID in namespace superiori

Global Namespace	PID = 24133	PID visto dal:	Memorizzato in array numbers[]
└─ Namespace A	PID = 1	Namespace B (corrente)	numbers[0] = 1
└─ Namespace B	PID = 1	Namespace A (genitore)	numbers[1] = 47
		Global namespace	numbers[2] = 24322

(struct pid tiene la "traduzione" dei PID lungo la gerarchia)

Si possono vedere in:

proc/<PID>/ns

Proc/<PID>/status

proc/<PID>/ns/net

Nspid: 24133 1 1

proc/<PID>/ns/pid ...

Container

■ Namespace Mount:

Mount namespace

Isola:

- La tabella dei mount point visibile da un processo
- La disposizione dei filesystem montati (/ , /proc, /sys, /home, ecc.)

Non isola:

- Il contenuto dei file
- I dati memorizzati sui filesystem reali
- Il dispositivo

(Le modifiche ai file reali rimangono reali, a meno di filesystem sovrapposti (es. overlayfs → Docker))

Un processo può:

- montare filesystem senza influenzare gli altri
- smontare filesystem solo per il suo namespace
- rimpiazzare parti dell'albero

Container

- Namespace NET:

NET namespace

Isola:

- L'intero **network stack** visto dal processo
- tabelle di routing
- Interfacce
- Indirizzi IP e porte TCP/UDP
- firewall
- stato delle connessioni

Non isola:

- la scheda di rete vera
- La banda reale disponibile
- Le connessioni disponibili

I namespace di rete hanno lo stesso codice TCP/IP ma istanze separate delle strutture dati

Due processi in network namespace differenti possono bindare la stessa IP e porta (nel namespace) senza conflitti.

Container

- `clone()`: È la primitiva che permette di creare un processo figlio con uno o più namespace “nuovi”.

```
int clone(  
    int (*fn)(void *),  
    void *child_stack,  
    int flags,  
    void *arg,  
    ...  
);
```

Si può passare a `clone` come flag:

- `CLONE_NEWNS` → mount namespace
- `CLONE_NEWPID` → PID namespace
- `CLONE_NEWUTS` → hostname
- `CLONE_NEWIPC` → IPC namespace
- `CLONE_NEWNET` → network namespace
- `CLONE_NEWUSER` → user namespace
- `CLONE_NEWCGROUP` → cgroup namespace

```
clone(child_func, stack, CLONE_NEWPID | CLONE_NEWNS | SIGCHLD, NULL);
```

Container

- `unshare()`: separa il processo corrente da un namespace”.

```
int unshare(int flags);
```

Esempio

```
unshare(CLONE_NEWNET | CLONE_NEWUTS);
```

Container

- `setns()`: entra in un namespace esistente

```
#define _GNU_SOURCE  
#include <sched.h>
```

```
int setns(int fd, int nstype);
```

Deve passare un file descriptor verso un namespace, ad esempio:

```
int fd = open("/proc/1234/ns/net", O_RDONLY);  
setns(fd, CLONE_NEWNET);
```

Container

- interfaccia file-based per i namespace

Ogni namespace è esposto come file speciale:

```
/proc/<pid>/ns/pid  
/proc/<pid>/ns/net  
/proc/<pid>/ns/user  
...
```

Si possono aprire questi file per usarli con `setns()`. Esempio:

```
int fd = open("/proc/1234/ns/pid", O_RDONLY);  
setns(fd, 0);
```

Container

- Mount + pivot

Ogni namespace è esposto come file speciale:

Il mount namespace da solo non basta per creare un FS privato.

Si usano syscall aggiuntive:

- mount() — montare FS in un mount
- namespaceumount2() — smontare
- pivot_root() — cambiare la radice del FS

Container

- **Riepilogo funzioni**

Funzione	A cosa serve	Standard
clone()	creare processo con nuovi namespace	Linux specifica
unshare()	creare nuovi namespace per il processo corrente	Linux specifica
setns()	entrare in namespace esistente	Linux specifica
/proc/PID/ns/*	file descriptor dei namespace	Linux specifico
mount(), pivot_root()	isolamento FS	POSIX + Linux extension

Non esiste una API POSIX per i namespace. I namespace sono una funzionalità Linux e si usano tramite syscall Linux-specifiche.

Container

■ Prova programma in C

- Compilare il test_container.c
 - `gcc -Wall -O2 test_container.c -o test_container`
- Eseguire
 - `sudo ./test_container`
- Messaggio:
 - [parent] PID del processo principale (host): 12345
 - [parent] Creo un 'mini container' con PID+UTS+MOUNT(+NET) namespace...
 - [parent] clone() ha creato un child con PID host: 12346
 - [parent] Dentro il nuovo PID namespace il child sarà PID 1.
- Poi è lanciata sh con output
 - [child] Sono nel child, PID visto da qui: 1
 - [child] Hostname nel namespace: nsdemo
 - [child] Esecuzione di /bin/sh dentro il namespace.
 - Prova questi comandi:
 - hostname
 - ps aux
 - ls /proc

Container

■ Prova programma in C

- Compilare il test_container.c
 - `gcc -Wall -O2 test_container.c -o test_container`
- Eseguire
 - `sudo ./test_container`
- Poi è lanciata sh con output

```
[child] Sono nel child, PID visto da qui: 1
[child] Hostname nel namespace: nsdemo
[child] Esecuzione di /bin/sh dentro il namespace.
Prova questi comandi:
- hostname
- ps aux
- ls /proc
```
- Eseguire:
 - Comandi indicati
 - In altra shell: `ps aux | grep test_container`

Control Group

- I cgroups (Control Groups) sono una funzionalità del kernel Linux che permette di limitare, monitorare e isolare l'uso delle risorse di un gruppo di processi.
- Con i namespace isoli *cosa* un processo può **vedere**.
- Con i cgroups isoli *quante* risorse può **consumare**.

- Presenti nel Kernel linux, introdotti da Google nel 2007
 - versioni v1 e v2 (più recente)
- Agiscono tramite uno pseudo-filesystem
 - Generalmente `/sys/fs/cgroup`

Control Group

- I cgroups (Control Groups) sono una funzionalità del kernel Linux che permette di limitare, monitorare e isolare l'uso delle risorse di un gruppo di processi.
- Agiscono tramite uno pseudo-filesystem
 - Generalmente `/sys/fs/cgroup`
- Esempio (limite memoria):
 - `sudo mkdir /sys/fs/cgroup/mygroup`
 - `echo 100M | sudo tee /sys/fs/cgroup/mygroup/memory.max`
 - La dir `mygroup` rappresenta un gruppo di processi con limite di memoria
 - Qui `tee` prende da `stdin` e scrive su file e `stdout`
 - `echo <PID> | sudo tee /sys/fs/cgroup/mygroup/cgroup.procs`
 - Aggiunge il processo al gruppo

Control Group

- I cgroups (Control Groups) sono una funzionalità del kernel Linux che permette di limitare, monitorare e isolare l'uso delle risorse di un gruppo di processi.
- Agiscono tramite uno pseudo-filesystem
 - Generalmente `/sys/fs/cgroup`
- Esempio (limite cpu):
 - `sudo mkdir /sys/fs/cgroup/mycpu`
 - `echo "20000 100000" | sudo tee /sys/fs/cgroup/mycpu/cpu.max`
 - quota = 20000 μ s (20 ms), period = 100000 μ s (100 ms)
 - La dir mycpu rappresenta un gruppo di processi con limite di cpu
 - `echo <PID> | sudo tee /sys/fs/cgroup/mygroup/cgroup.procs`
 - Aggiunge il processo al gruppo

Control Group

- I cgroups (Control Groups) sono una funzionalità del kernel Linux che permette di limitare, monitorare e isolare l'uso delle risorse di un gruppo di processi.
- Agiscono tramite uno pseudo-filesystem
 - Generalmente `/sys/fs/cgroup`
- Esempio (limite processi):
 - `sudo mkdir /sys/fs/cgroup/mylimit`
 - `echo 50 | sudo tee /sys/fs/cgroup/mylimit/pids.max`
 - La dir `mylimit` rappresenta un gruppo di processi con limite di pid
 - `echo <PID> | sudo tee /sys/fs/cgroup/mygroup/cgroup.procs`
 - Aggiunge il processo al gruppo
 - `echo max | sudo tee /sys/fs/cgroup/mylimit/pids.max`
 - per ripristinare

Control Group

- Nel kernel ogni cgroup è rappresentato da una struttura:

```
struct cgroup {  
    struct kernfs_node *kn; // entry nel filesystem cgroup  
    struct cgroup *parent; // padre nella gerarchia  
    atomic_t nr_descendants;  
    ...  
};
```

- Ogni processo (task_struct) contiene questa informazione:

```
struct task_struct {  
    ...  
    struct css_set *cgroups; // insieme degli stati cgroup del task  
    ...  
};
```

- Caratteristiche:

- Attraverso struct css_set *cgroups il task risale al cgroup del processo
- un cgroup è un nodo dell'albero
- esiste una gerarchia ad albero: padre → figli → nipoti → ...
- in cgroup v2 **ogni task appartiene a un solo cgroup** della gerarchia unificata
- il kernel usa questo per applicare limiti (memory.max, cpu.max, ...)

Control Group

- In user space i cgroup v2 si gestiscono semplicemente scrivendo file in `/sys/fs/cgroup`. In C significa: `mkdir()`, `open()`, `write()`, `close()`

- Esempio:
 - Programma C che:
 - controlla che esista `/sys/fs/cgroup` (cgroup v2 montato in unificato)
 - crea una directory `/sys/fs/cgroup/demo_cgroup`
 - fa `fork()`
 - sposta il figlio nel cgroup scrivendo il suo PID in `cgroup.procs`
 - imposta un limite di memoria `memory.max`
 - nel figlio prova ad allocare memoria in un loop
 - Testare:
 - `gcc -Wall -O2 test_cgroup.c -o test_cgroup`
 - Provare `mount | grep cgroup2`
 - Lanciare: `sudo ./test_cgroup`

Control Group

- In user space i cgroup v2 si gestiscono semplicemente scrivendo file in `/sys/fs/cgroup`. In C significa: `mkdir()`, `open()`, `write()`, `close()`
- Esempio:
 - Testare:
 - `gcc -Wall -O2 test_cgroup.c -o test_cgroup`
 - Provare `mount | grep cgroup2`
 - Lanciare: `sudo ./test_cgroup`

```
[parent] cgroup creato: /sys/fs/cgroup/demo_cgroup
[parent] PID figlio: 12345
[child] PID figlio (host): 12345
[child] Attendo che il padre mi metta nel cgroup...
[parent] Spostato PID 12345 in /sys/fs/cgroup/demo_cgroup
[parent] Impostato memory.max = 50MB per
/sys/fs/cgroup/demo_cgroup
[child] Provo ad allocare memoria in un loop...
[child] allocati ~10 MB
[child] allocati ~20 MB
...
```

Control Group

- In user space i cgroup v2 si gestiscono semplicemente scrivendo file in `/sys/fs/cgroup`. In C significa: `mkdir()`, `open()`, `write()`, `close()`

- Esempio:
 - Testare:
 - `gcc -Wall -O2 test_cgroup.c -o test_cgroup`
 - Provare `mount | grep cgroup2`
 - Lanciare: `sudo ./test_cgroup`

 - Dall'host provare
 - `cat /sys/fs/cgroup/demo_cgroup/memory.current`
 - `cat /sys/fs/cgroup/demo_cgroup/cgroup.procs`

Control Group

- In user space i cgroup v2 si gestiscono semplicemente scrivendo file in `/sys/fs/cgroup`. In C significa: `mkdir()`, `open()`, `write()`, `close()`

- Esempio:
 - Notare:
 - Non esiste una syscall “`set_cgroup`”:
il kernel espone un filesystem e tu scrivi file da C (o da qualsiasi linguaggio).
 - Il legame task → cgroup avviene scrivendo il PID in `cgroup.procs`.
 - I limiti (es. `memory.max`) sono solo file di testo in `/sys/fs/cgroup/....`
 - È lo stesso meccanismo che userà `Docker/run`

Overlay Filesystems

- OverlayFS è un **filesystem “a strati” (union filesystem)** usato dai container per combinare:
 - uno o più **layer in sola lettura** (immagini)
 - un **layer in scrittura** (container)
- Il risultato è un singolo filesystem “virtuale” che sembra completo.
- Un container deve:
 - avere il suo filesystem “privato”, senza toccare quello dell’host
 - essere leggibile e veloce da creare
 - evitare duplicazioni inutili
 - permettere immutabilità del sistema di base
 - Per ottenere questo i container usano **OverlayFS**, un filesystem del kernel Linux.

Overlay Filesystems

- OverlayFS è un **filesystem “a strati” (union filesystem)** usato dai container per combinare:
 - uno o più **layer in sola lettura** (immagini)
 - un **layer in scrittura** (container)
- Il risultato è un singolo filesystem “virtuale” che sembra completo.
- Strati:
 - lowerdir = layer immagine (RO)
 - upperdir = layer del container (RW)
 - workdir = area di lavoro temporanea del kernel
 - merged = vista finale (RO+RW)

Overlay Filesystems

- OverlayFS è un **filesystem “a strati” (union filesystem)** usato dai container per combinare:
 - uno o più **layer in sola lettura** (immagini)
 - un **layer in scrittura** (container)
- **Principio fondamentale: Copy-on-Write**
 - Quando il container modifica un file:
 - Il file **non viene cambiato** nel layer immagine (lowerdir, RO)
 - OverlayFS **copia il file** nell'upperdir (RW)
 - La VM / container vede solo la versione modificata
 - **Le modifiche restano isolate nel layer superiore senza mai toccare l'immagine originale in sola lettura.**
 - **Vantaggi**
 - Avvio velocissimo (le immagini sono RO, condivise tra container)
 - Poco spazio occupato (solo le modifiche sono salvate)
 - Rollback immediato (basta cancellare l'upperdir)
 - Massima riproducibilità (la base non cambia mai)

Overlay Filesystems

■ Esempio

- mkdir lower upper work merged
- echo "Hello DAL LOWER" > lower/file.txt
- sudo mount -t overlay overlay \
- o lowerdir=lower,upperdir=upper,workdir=work merged
- cat merged/file.txt //hello dal lower
- echo "MODIFICATO DAL CONTAINER" > merged/file.txt
- cat upper/file.txt # <-- Esiste ed è modificato
- cat lower/file.txt # <-- Rimane invariato
- rm merged/file.txt # solo per upper è cancellato
- ls -a upper # si vede .wh.file.txt, è whiteout cioè nasconde il file lower
- sudo umount merged
- rm -rf lower upper work merged